WEB-BASED ML MODEL EVALUATOR

Code Coverage Report

Project	Web-Based ML Model Evaluator (Team SE-Jackfruit)
Backlog item	BL-15 - Code Coverage and Branch Coverage Reports
Result	Line (statement) coverage 95.8% - 1,063 of 1,110 statements run

1. Purpose

Code coverage shows how much of the program's code was run while the tests were running. It answers: *which parts of the code did the tests reach, and which parts did they never touch?* A statement that no test runs is a statement whose behaviour has not been checked. This report measures **line (statement) coverage**. Decision paths inside a line are covered in the separate Branch Coverage Report.

The numbers come from coverage.py, the standard Python coverage tool, driven by pytest-cov while the same 219 tests of BL-13 ran. No test was added or changed for this report.

2. Method and environment

Item	Detail
Test run	5 November 2026, 16:46 IST (timestamp in the coverage file)
Environment	Windows, Python 3.14.6, pytest 9.1.1, pytest-cov 7.1.0, coverage.py 7.17.2
Command	python -m pytest tests -q -p no:cacheprovider --cov=app --cov=config --cov=services --cov-branch --cov-report=term-missing --cov-report=json --cov-report=xml --cov-report=html
Tests run	The full suite: 219 tests, all passed (63.1 s)
Code measured	The application code only: app.py, config.py and the services folder (15 files, 1,110 statements, 270 branches). The test files themselves are not measured.
Evidence files	coverage-output.txt, coverage.json and coverage.xml in docs/test-results/coverage

3. Overall result

Statements	Run by tests	Not run	Line coverage
1,110	1,063	47	95.8%

Of 1,110 executable statements in the application, 1,063 ran at least once (95.8%). 7 of the 14 files with statements are at or above 95 %, and 3 are fully covered (errors.py, model_service.py, timeutil.py). The tool's own single 'total' of 94 % in the terminal output blends lines and branches together; this report states line coverage on its own.

4. Coverage by file

File	Statements	Run	Not run	Line coverage	Lines not run
app.py	345	335	10	97.1%	125, 140-142, 167, 409-410, 496-497, 502
config.py	28	25	3	89.3%	41, 44-45
services/__init__.py	0	0	0	n/a	none
services/admin_service.py	54	53	1	98.1%	32
services/auth_service.py	62	59	3	95.2%	29, 42, 63
services/compatibility.py	16	15	1	93.8%	27
services/dataset_service.py	84	79	5	94.0%	105-107, 111-112
services/db.py	59	56	3	94.9%	115-117
services/errors.py	3	3	0	100.0%	none
services/evaluation_service.py	115	106	9	92.2%	32-33, 44-45, 96-97, 102, 108, 145
services/metrics.py	24	22	2	91.7%	42-43
services/model_service.py	69	69	0	100.0%	none
services/report_service.py	73	69	4	94.5%	46, 132-134
services/timeutil.py	3	3	0	100.0%	none
services/visualization_service.py	175	169	6	96.6%	55-59, 80
Total	1,110	1,063	47	95.8%	

5. What the tests did not reach

Every statement that no test ran is listed below with its purpose. They are mostly safety nets for rare errors, start-up code and double checks of rules that are already enforced earlier in the program.

File	Lines	What the code does
app.py	125	Generic 'something went wrong' page for browser (non-API) requests. The API form of the 500 error is tested.
app.py	140-142	The home address '/' redirecting to the login page or to the user's own home page.
app.py	167	Register page opened by someone who is already logged in (redirects to their home page).
app.py	409-410	Error response of the compare-chart image when the selection is invalid. The same checks are tested on the compare table.
app.py	496-497	Error message of the create-admin command when the input is rejected. The successful command is tested.
app.py	502	Start-up line used only when app.py is run directly. The project is started with 'flask run'.
config.py	41	Secret key read from the EVALUATOR_SECRET_KEY environment variable.
config.py	44-45	First-time creation of the secret-key file. Later starts read the existing file.
services/admin_service.py	32	Safety check that only an active Admin may manage accounts. The web routes already block everyone else (tested in SEC_06 and SEC_07).
services/auth_service.py	29	Helper that returns nothing when asked for the public view of a missing user.
services/auth_service.py	42	Name longer than 80 characters is rejected.
services/auth_service.py	63	A role other than analyst or admin is rejected inside the service.
services/compatibility.py	27	A model that states neither its feature names nor its feature count is accepted as compatible.
services/dataset_service.py	105-107	The stored dataset file cannot be re-read at validation time (file damaged after upload).
services/dataset_service.py	111-112	Target column missing from the stored file at validation time (it is already checked at upload).
services/db.py	115-117	Database rollback if a write fails halfway.
services/evaluation_service.py	32-33	The stored model file cannot be loaded when an evaluation starts.
services/evaluation_service.py	44-45	The stored dataset file cannot be read when an evaluation starts.
services/evaluation_service.py	96-97	The model's probability function fails during evaluation: ROC-AUC is skipped with a note.
services/evaluation_service.py	102	ROC-AUC cleared after the probability function failed (follows the case above).
services/evaluation_service.py	108	A validation error inside the evaluation step is passed on unchanged.
services/evaluation_service.py	145	Public view of a missing evaluation returns nothing.
services/metrics.py	42-43	ROC-AUC calculation raises an error: a note is added instead of failing.
services/report_service.py	46	A missing metric value is printed as 'Not available' in a report.
services/report_service.py	132-134	The 'Notes' section of a report, printed when an evaluation has notes.
services/visualization_service.py	55-59	Conversion of numpy decimal and true/false values in chart data (the other value types are used in tests).
services/visualization_service.py	80	A class that is absent from the data is skipped when drawing ROC curves.

6. Assessment

Core features are well covered. The login, roles, data isolation, model and dataset handling, metric calculation, charts, comparison and PDF reports are all at 91 % or above. The statements not run are not part of the main user journeys.

Lowest file: config.py (89.3 %). The two gaps are the secret-key setup: the environment-variable route and the first-run file creation. No automated test covers them.

Why the gaps were left. Reaching them needs damaged stored files, failures in the middle of database writes or unusual start-up situations, which are awkward to create in an automated test. The tests were not changed for this report so that the results of BL-13 and BL-14 (219 tests, 86 of 86 test cases) stay exactly the same.

Limit of the measure. A statement being run does not prove it behaves correctly. Coverage shows what was reached; the test results show whether it was right.

7. Conclusion

Line coverage of 95.8% shows that the automated tests exercise almost all of the application's code, including every main feature. The remaining statements are listed above with their purpose. If time allows later, small tests for the name-length rule, the role rule and the create-admin error message would close several of them cheaply.